



UNIVERSITAT ROVIRA I VIRGILI

Detección de Fraude en Transacciones Bancarias mediante Técnicas Clásicas de Machine Learning Clásico

Artificial Intelligence Foundations

- KEVIN ALEXANDER ROMAN ZARUMA
- EMANUEL NESTOR ESCOBAR JUIPA



Introducción

- El fraude financiero representa un desafío creciente.
- Machine Learning permite detectar patrones anómalos.

Objetivo: Detectar automáticamente transacciones fraudulentas de la base de datos de Detección de fraude de Tarjetas de Crédito de Kaggle.

Formulación del problema

Tipo de aprendizaje: Supervisado

Tipo de tarea: Clasificación binaria

Variable objetivo:

El problema corresponde a una tarea de clasificación binaria, donde el modelo debe asignar cada transacción a una de dos clases posibles: fraudulenta o legítima.

Clases (binario)

0 = legítima

1 = fraudulenta

Carga y exploración del dataset

El **dataset** contiene:

- 284.807 registros
- 31 columnas

Entre las **variables principales** se encuentran:

- Time
- Amount
- Las variables V1–V28 corresponden a componentes principales obtenidos mediante PCA para preservar la confidencialidad de los datos originales.
- Class (variable objetivo)

Visualización de datos

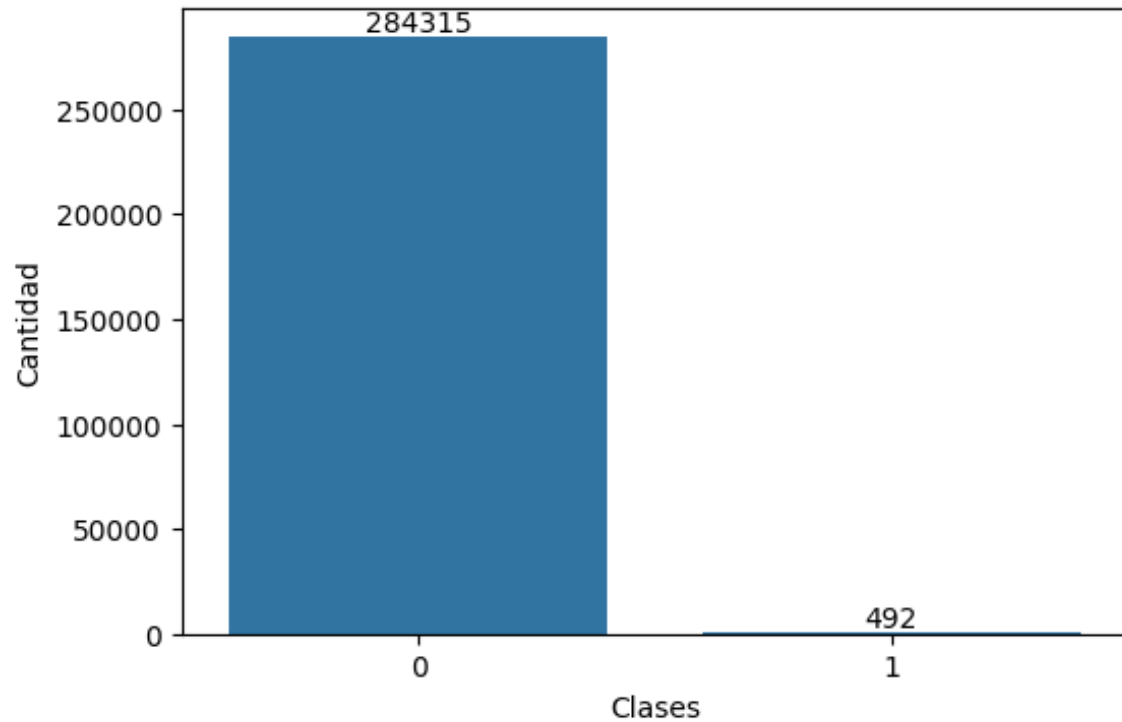


Figura 1. Distribución de Transacciones Legítimas y Fraudulentas

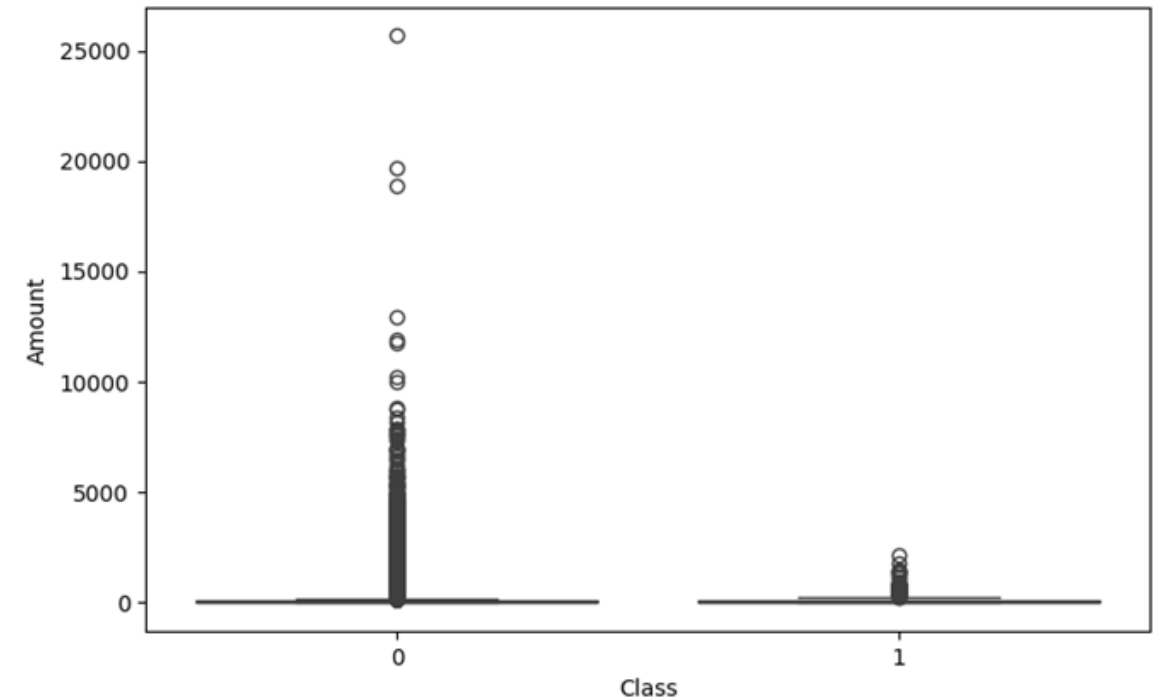


Figura 2. Importe de la transacción por clase

Preprocesamiento de datos

División del dataset

test_size=0.2

80% entrenamiento

20% evaluación

Escalado de variables

Las variables **Time y Amount** fueron normalizadas utilizando **StandardScaler**.

El **escalado** permitió normalizar las variables numéricas y evitar sesgos derivados de diferencias de magnitud entre atributos.

El **scaler** se ajustó únicamente sobre los datos de entrenamiento para **evitar data leakage**.

Verificar balance de clases (SMOTE)

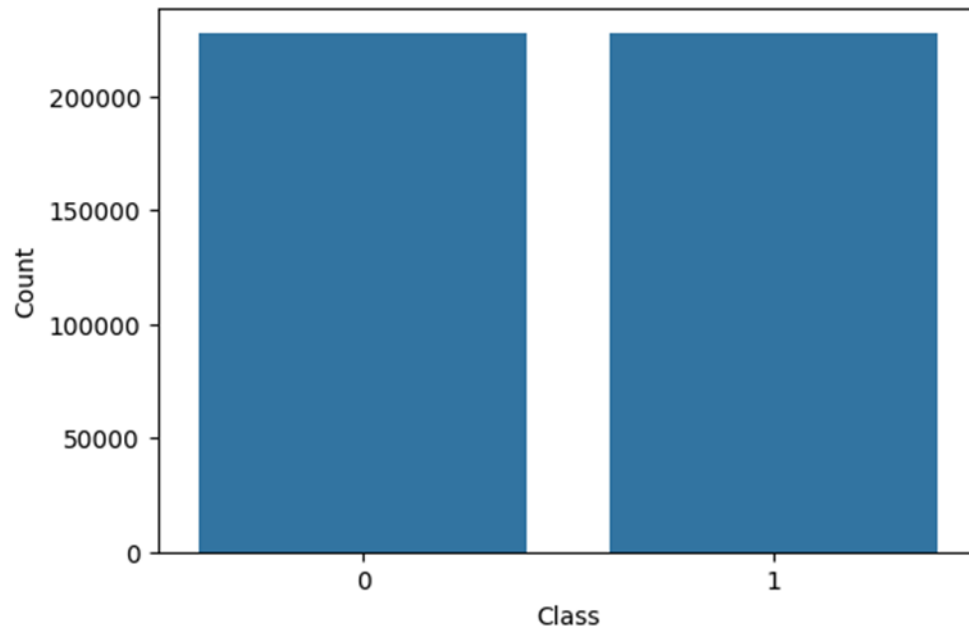


Figura 3. Balance de clases después de usar SMOTE

El **SMOTE** generó muestras sintéticas de la clase minoritaria utilizando vecinos cercanos, permitiendo balancear el conjunto de entrenamiento y mejorar la capacidad predictiva del modelo.

Entrenamiento del modelo

Se entrenó un modelo de clasificación utilizando **Logistic Regression** and **Random Forest**.

Logistic Regression: fue seleccionada como modelo base debido a su simplicidad, interpretabilidad y eficiencia en problemas de clasificación binaria.

```
class_weight='balanced'
```

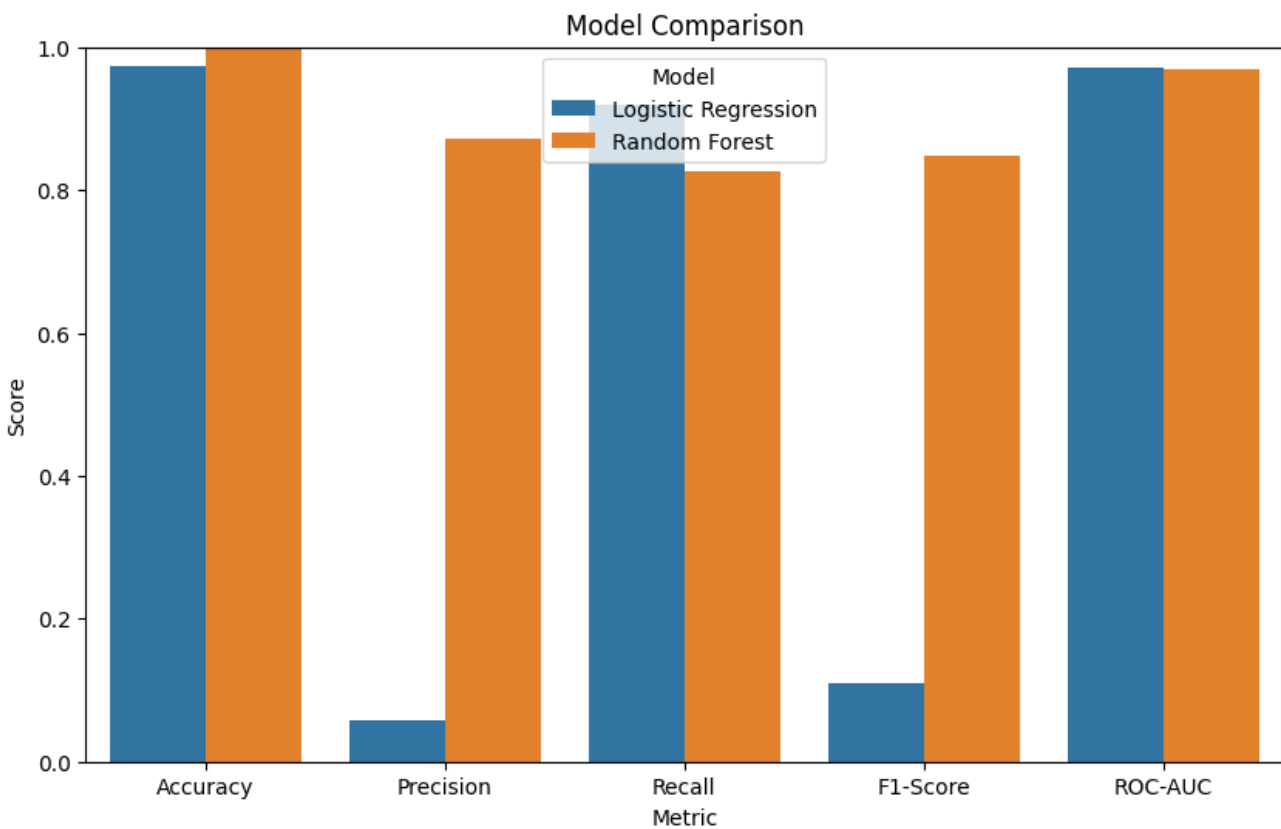
```
max_iter=1000
```

Random Forest: fue seleccionado por su capacidad para manejar relaciones no lineales y reducir overfitting mediante ensamblaje de árboles.

```
n_estimators=100
```

```
class_weight='balanced'
```


Comparación de modelos



Modelo	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	97.55%	6.09%	91.84%	10.88%	97.08%
Random Forest	99.95%	96.05%	74.49%	84.82%	96.85%

El mejor modelo fue
Random Forest

Figura 4. Comparacion entre Logistic Regression y Random Forest

Ajuste de Hiperparámetros

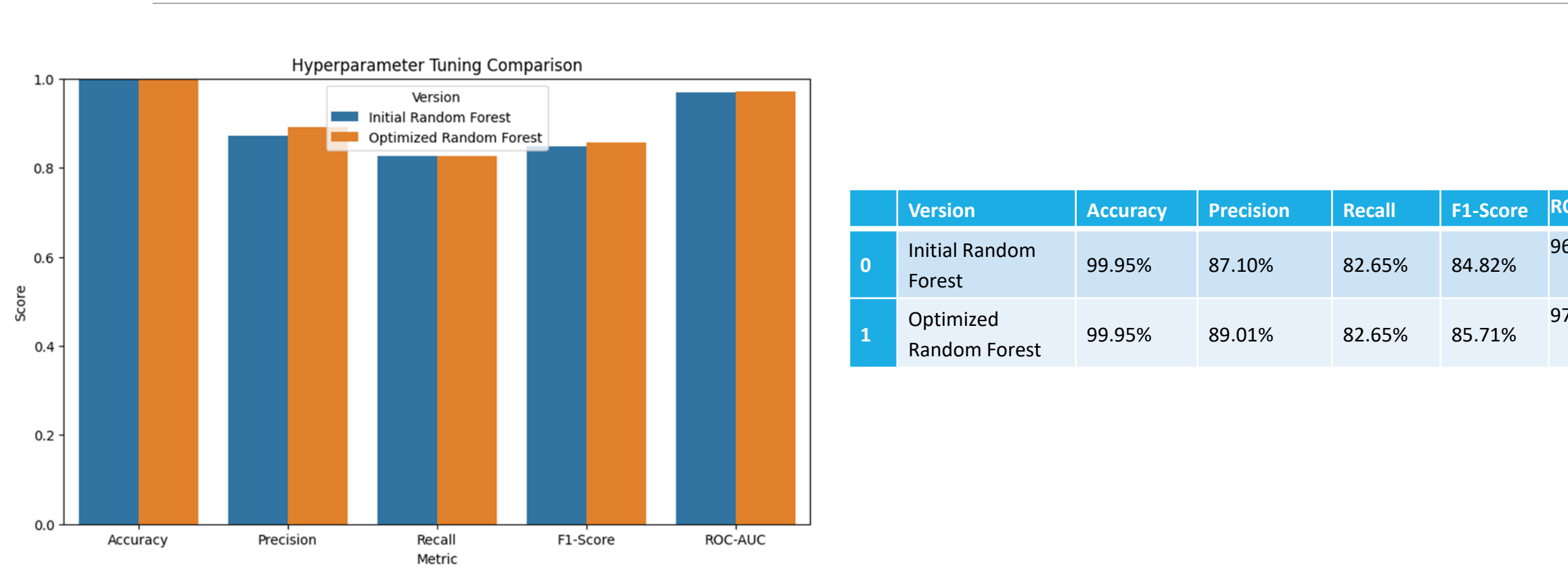
GridSearchCV de la librería de sklearn.

Permitió evaluar sistemáticamente múltiples combinaciones de hiperparámetros mediante **validación cruzada**.

Uso de parámetros como `n_estimators` (número de árboles), `max_depth` (profundidad máxima), `min_sample_split` (mínimo para dividir nodos) y `min_sample_leaf`

```
param_grid = {  
    'n_estimators': [100, 200],  
    'max_depth': [10, None],  
    'min_samples_split': [2,4],  
    'min_samples_leaf': [1,2]  
}  
scoring='f1',  
cv=2,
```

El proceso permitió evaluar múltiples combinaciones (n=32) de parámetros utilizando validación cruzada



	Version	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Initial Random Forest	99.95%	87.10%	82.65%	84.82%	96.85%
1	Optimized Random Forest	99.95%	89.01%	82.65%	85.71%	97.08%

Figura 5. Comparación entre RF inicial y RF optimizado

Conclusiones

1. Se desarrolló un sistema de detección de fraude utilizando ML sobre 284,807 transacciones bancarias, identificando un fuerte desbalance de clases (0.17% fraudes).
2. Logistic Regression alcanzó un recall de 91.84%, detectando la mayoría de fraudes, aunque con muchos falsos positivos.
3. Random Forest obtuvo un desempeño más equilibrado, con 99.95% de accuracy y 96.05% de precisión, reduciendo significativamente falsas alarmas.
4. La optimización de hiperparámetros mejoró el rendimiento del modelo, aumentando precision (87.10% → 89.10%), F1-score (84.82% → 85.71%) y ROC-AUC (96.85% → 97.08%).